

PRACTICAL – 1

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <chrono>
```

```
using namespace std;
```

```
using namespace std::chrono;
```

```
// 1. Bubble Sort
```

```
void sortBubble(vector<int> &nums) {
```

```
    int len = nums.size();
```

```
    for (int pass = 0; pass < len - 1; pass++) {
```

```
        for (int idx = 0; idx < len - pass - 1; idx++) {
```

```
            if (nums[idx] > nums[idx + 1]) {
```

```
                swap(nums[idx], nums[idx + 1]);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
// 2. Selection Sort
```

```
void sortSelection(vector<int> &nums) {
```

```
    int len = nums.size();
```

```
    for (int step = 0; step < len - 1; step++) {
```

```
        int smallest = step;
```

```
        for (int curr = step + 1; curr < len; curr++) {
```

```
            if (nums[curr] < nums[smallest]) {
```

PRACTICAL – 1

```
        smallest = curr;
    }
}
swap(nums[step], nums[smallest]);
}
}
```

// 3. Insertion Sort

```
void sortInsertion(vector<int> &nums) {
    int len = nums.size();
    for (int step = 1; step < len; step++) {
        int currentVal = nums[step];
        int pos = step - 1;

        while (pos >= 0 && nums[pos] > currentVal) {
            nums[pos + 1] = nums[pos];
            pos--;
        }
        nums[pos + 1] = currentVal;
    }
}
```

// 4. Merge Sort

```
void combineParts(vector<int> &nums, int left, int mid, int right) {
    vector<int> tempBuffer;
    int p1 = left, p2 = mid + 1;

    // Compare and merge into a single temporary buffer
```

PRACTICAL – 1

```
while (p1 <= mid && p2 <= right) {
    if (nums[p1] <= nums[p2]) {
        tempBuffer.push_back(nums[p1++]);
    } else {
        tempBuffer.push_back(nums[p2++]);
    }
}

// Add any remaining elements
while (p1 <= mid) tempBuffer.push_back(nums[p1++]);
while (p2 <= right) tempBuffer.push_back(nums[p2++]);

// Copy back to the original array
for (int idx = 0; idx < tempBuffer.size(); idx++) {
    nums[left + idx] = tempBuffer[idx];
}
}

void sortMerge(vector<int> &nums, int start, int end) {
    if (start < end) {
        int center = start + (end - start) / 2;
        sortMerge(nums, start, center);
        sortMerge(nums, center + 1, end);
        combineParts(nums, start, center, end);
    }
}

// 5. Quick Sort
```

PRACTICAL – 1

```
int divideArray(vector<int> &nums, int start, int end) {  
    int pivotVal = nums[end];  
    int boundary = start;  
  
    for (int curr = start; curr < end; curr++) {  
        if (nums[curr] < pivotVal) {  
            swap(nums[boundary], nums[curr]);  
            boundary++;  
        }  
    }  
    swap(nums[boundary], nums[end]);  
    return boundary;  
}
```

```
void sortQuick(vector<int> &nums, int start, int end) {  
    if (start < end) {  
        int pivotIdx = divideArray(nums, start, end);  
        sortQuick(nums, start, pivotIdx - 1);  
        sortQuick(nums, pivotIdx + 1, end);  
    }  
}
```

```
int main() {  
    const int totalItems = 100;  
    vector<int> mainArray(totalItems), copyArray;  
  
    srand(time(0));
```

PRACTICAL – 1

```
// Fill array with random numbers
for (int x = 0; x < totalItems; x++) {
    mainArray[x] = rand() % 1000;
}

cout << "Number of Elements = " << totalItems << "\n\n";

// Timing variables
auto beginTimer = high_resolution_clock::now();
auto endTimer = high_resolution_clock::now();

// Test Bubble Sort
copyArray = mainArray;
beginTimer = high_resolution_clock::now();
sortBubble(copyArray);
endTimer = high_resolution_clock::now();

cout << "Bubble Sort Time   : " << duration_cast<microseconds>(endTimer -
beginTimer).count() << " microseconds\n";

// Test Selection Sort
copyArray = mainArray;
beginTimer = high_resolution_clock::now();
sortSelection(copyArray);
endTimer = high_resolution_clock::now();

cout << "Selection Sort Time : " << duration_cast<microseconds>(endTimer -
beginTimer).count() << " microseconds\n";

// Test Insertion Sort
```

PRACTICAL – 1

```
copyArray = mainArray;

beginTimer = high_resolution_clock::now();

sortInsertion(copyArray);

endTimer = high_resolution_clock::now();

cout << "Insertion Sort Time : " << duration_cast<microseconds>(endTimer -
beginTimer).count() << " microseconds\n";


// Test Merge Sort

copyArray = mainArray;

beginTimer = high_resolution_clock::now();

sortMerge(copyArray, 0, totalItems - 1);

endTimer = high_resolution_clock::now();

cout << "Merge Sort Time   : " << duration_cast<microseconds>(endTimer -
beginTimer).count() << " microseconds\n";


// Test Quick Sort

copyArray = mainArray;

beginTimer = high_resolution_clock::now();

sortQuick(copyArray, 0, totalItems - 1);

endTimer = high_resolution_clock::now();

cout << "Quick Sort Time   : " << duration_cast<microseconds>(endTimer -
beginTimer).count() << " microseconds\n";


return 0;

}
```